Design and Implementation of an End-to-End DSL Compiler

Course: Compiler Design
Name: Rithanya M
Registration Number: RA2311026050117
Department: BTech CSE AIML - D
Institution: SRM Institute of Science and Technology

1. Project Overview

This project presents the design and implementation of a simplified **End-to-End Compiler for a Domain-Specific Language (DSL)**. The system demonstrates the core phases of compiler construction in a modular and easy-to-understand manner.

The compiler processes basic assignment statements and arithmetic expressions, transforming them into an intermediate representation through multiple compilation stages.

2. Objectives

- To simulate the complete compiler pipeline
- To process simple DSL statements efficiently
- To demonstrate lexical, syntactic, and semantic analysis
- To generate intermediate code representation
- To build a structured and modular compiler design

3. Compiler Phases Implemented

i. Lexical Analysis

The lexical analyzer identifies tokens from the input source code.

Tokens recognized include:

- Identifiers (x, y, z)
- Numeric constants (5, 10)
- Operators (+, =)

ii. Syntax Analysis

The parser validates whether the input follows correct grammar rules.

Example:
z = x + y

iii. Abstract Syntax Tree (AST) Construction

The AST represents the hierarchical structure of expressions.
It simplifies further processing by organizing operations logically.

iv. Semantic Analysis

This phase checks the correctness of the program logic.

It ensures:

- Variables are valid
- Expressions are meaningful
- Assignments are correct

v. Intermediate Code Generation

The compiler generates a simplified intermediate representation.

Example:
temp = x + y
z = temp

4. Sample Input

x = 5
y = 10
z = x + y

5. Sample Output

[Lexer] Identifier token detected
[Lexer] Number token detected

[Parser] Assignment statement recognized
[Semantic] Valid variable assignment
[CodeGen] Intermediate code: temp = x + y

6. Technologies Used

- C Programming Language
- Flex (Lexical Analysis)
- Bison (Parsing)
- GCC Compiler

7. Project Structure

Compiler-Design-DSL/
├── src/ → Source code files
├── test/ → Input file
├── output/ → Execution screenshots
├── docs/ → Project report
└── README.md

8. Execution Steps

1. Open terminal in the src directory
2. Run Bison:
   bison -d parser.y
3. Run Flex:
   flex lexer.l
4. Compile using GCC:
   gcc lex.yy.c parser.tab.c ast.c semantic.c codegen.c -o compiler
5. Execute program:
   ./compiler < ../test/input.txt

9. Key Features

- Modular compiler architecture

- Clear demonstration of compilation stages
- Structured and readable output
- Easy to extend for advanced compiler concepts

10. Conclusion

This project successfully demonstrates how a compiler processes input step-by-step through multiple phases. It provides a strong foundation for understanding compiler design concepts and highlights the importance of modular design in building scalable systems.

Done By:
Rithanya M
RA2311026050117